

Lista 13 - Exercícios de Lista Duplamente Encadeada

1. Implemente funções para as seguintes operações em uma lista duplamente encadeada:

- Inserir no final da lista
- Remover o primeiro elemento
- Remover o último elemento. **É proibido percorrer a lista. Para encontrar o penúltimo nó, acesse o ponteiro “anterior”.**

Observação: faça uso do descritor da lista para remover ou inserir um novo nó, ou seja, não é necessário percorrer a lista.

2. Crie as seguintes funções

- RemoveFim – Remove o último elemento da lista. É proibido percorrer a lista ou utilizar listas auxiliares. Usar o ponteiro “ant” do último nó na remoção.

•

3. Implemente funções para as seguintes operações em uma lista duplamente encadeada:

```
//#remove da lista todas as ocorrências do parâmetro valor
```

```
void removeTodos(Lista *L, int valor);
```

```
//#Retorna uma nova lista, ordenada, com os K maiores elementos de L
```

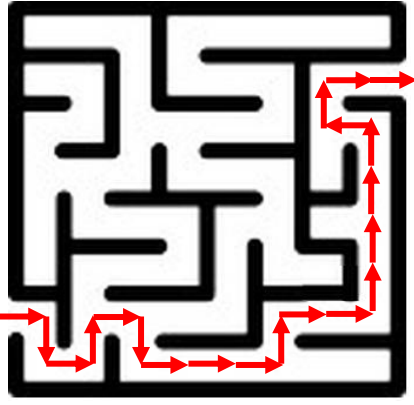
```
//#Se K for maior que o tamanho de L, a lista retornada deve conter todos os
```

```
// elementos de L de forma ordenada.
```

```
Lista leMaiores(Lista *L, int K);
```

4. Considere um labirinto, contendo um ponto de entrada e um ponto de saída. Dado um percurso do ponto de entrada do labirinto até um ponto saída (caminho de ida), escreva um programa em C++ para computar o percurso inverso (caminho de volta).

a. Exemplo dos percursos de ida e volta:

Caminho de ida	Caminho de volta	
direita, baixo, direita, cima, direita, baixo, direita, direita, direita, cima, direita, direita, cima, cima, cima, cima, esquerda, cima, direita, direita.	esquerda, esquerda, baixo, direita, baixo, baixo, baixo, baixo, esquerda, esquerda, baixo, esquerda, esquerda, esquerda, cima, esquerda, baixo, esquerda, cima, esquerda.	

b. Implemente o programa usando uma lista duplamente encadeada.

c. Exemplo de entrada/saída.

```
IDA: direita, baixo, direita, cima, direita, baixo, direita, direita, direita, cima, direit
a, direita, cima, cima, cima, cima, esquerda, cima, direita, direita

VOLTA: esquerda, esquerda, baixo, direita, baixo, baixo, baixo, baixo, esquerda, esquerda,
baixo, esquerda, esquerda, esquerda, cima, esquerda, baixo, esquerda, cima, esquerda
```

5. Desenvolva uma função em C++ que inverta a ordem dos N últimos elementos de uma Lista Duplamente Encadeada com Descritor.

O protótipo da função deve ser:

```
void inverterNFim(Lista* l, int n);
```

Observações:

- A lista deve armazenar números inteiros.
- O descritor da lista é composto por um ponteiro para o primeiro nó, um ponteiro para o último nó e o número de elementos armazenados.
- É permitido o uso de estruturas de dados auxiliares;
- Se o valor de N for maior que o número de elementos da lista, a operação deve ser abortada.
- Após a chamada da função inverterNFim, o programa principal deve exibir a lista tanto em ordem direta quanto em ordem inversa.

Dica de implementação:

- Acesse o último nó por meio do ponteiro fim (do descritor) e percorra os demais nós utilizando o ponteiro anterior de cada nó.

Exemplo de entrada e saída:

Um exemplo de entrada e saída são apresentados a seguir. O programa desenvolvido deve ser capaz de produzir a saída no formato mostrado.

```
#include <iostream>
using namespace std;
#include "lista_dupla.hpp"

int main(void) {
    setlocale(LC_ALL, "Portuguese");
```

```
cout << "\n#TESTE 1\n";
Lista lista = Lista();
insereInicioL(&lista, 9);
insereInicioL(&lista, 8);
insereInicioL(&lista, 7);
insereInicioL(&lista, 6);
insereInicioL(&lista, 5);
insereInicioL(&lista, 4);
insereInicioL(&lista, 3);
insereInicioL(&lista, 2);
insereInicioL(&lista, 1);
mostraL(&lista);
```

```
inverterNFim(&lista, 4);
mostraL(&lista);
mostrarInversoL(&lista);
```

```
cout << "\n#TESTE 2\n";
destróiL(&lista);
lista = Lista();
insereInicioL(&lista, 90);
insereInicioL(&lista, 80);
insereInicioL(&lista, 70);
insereInicioL(&lista, 60);
insereInicioL(&lista, 50);
insereInicioL(&lista, 40);
insereInicioL(&lista, 30);
insereInicioL(&lista, 20);
insereInicioL(&lista, 10);
mostraL(&lista);
```

```
inverterNFim(&lista, lista.tamanho);
mostraL(&lista);
mostrarInversoL(&lista);
```

```
destróiL(&lista);
return EXIT_SUCCESS;
```

#TESTE 1

L[9]:{1, 2, 3, 4, 5, 6, 7, 8, 9}

L[9]:{1, 2, 3, 4, 5, 9, 8, 7, 6}

L[9]:{6, 7, 8, 9, 5, 4, 3, 2, 1}

#TESTE 2

L[9]:{10, 20, 30, 40, 50, 60, 70, 80, 90}

L[9]:{90, 80, 70, 60, 50, 40, 30, 20, 10}

L[9]:{10, 20, 30, 40, 50, 60, 70, 80, 90}

6. Considere a manipulação de uma lista duplamente encadeada que armazene registros do tipo *Reuniao*, contendo os campos título e duração (em minutos). Desenvolva uma função em C++ que percorra a lista de reuniões e, com base na duração de cada reunião, realize a divisão das reuniões longas conforme os critérios especificados a seguir:

- a. A lista deve armazenar valores da struct *Reuniao* apresentada abaixo:

```
struct Reuniao
{
    string titulo;
    int duracao; //minutos
};
```

- b. Os descritores das listas devem ser: ponteiro para o primeiro nó, ponteiro para o último nó e o número de elementos armazenados na lista.
- c. A função deve identificar reuniões cuja duração exceda 240 minutos e, nesses casos, dividir a reunião em duas partes:
- A primeira parte permanece no nó original, com duração ajustada para exatamente 240 minutos;
 - A segunda parte, contendo o tempo restante, deve ser inserida imediatamente após a reunião original, como um novo nó na lista;
- d. O protótipo da função deve ser:
- ```
void dividirReunioesLongas (Lista *lista);
```
- e. A operação deve preservar a integridade da lista duplamente encadeada, atualizar corretamente seus ponteiros e manter os **descritores consistentes**;
- f. Não é permitido o uso de estrutura de dados auxiliares;

7. Considere as listas duplamente encadeadas com descritores L1, L2 e L3, que armazenam números inteiros. Os descritores dessas listas são compostos por: um ponteiro para o primeiro nó, um ponteiro para o último nó e o número de elementos armazenados. Implemente uma função que concatene as listas L1 e L2, armazenando o resultado em L3.

**O protótipo da função deve ser:**

```
void concatenarListas(Lista *l1, Lista *l2, Lista *l3);
```

**Observações:**

- Não concatene as listas caso L1 ou L2 esteja vazia.
- Não é permitido o uso de estruturas de dados auxiliares;

- Não é permitido o uso de laços de repetição;
- Os nós da lista L3 são os mesmos das listas L1 e L2. Ou seja, o primeiro nó de L3 é o primeiro nó de L1, e o último nó de L3 é o último nó de L2. Além disso, é necessário ajustar os ponteiros *anterior* e *próximo* para conectar o último nó de L1 ao primeiro nó de L2.
- Após a operação de concatenação das listas, L1 e L2 devem ficar vazias. Ou seja, os ponteiros para o primeiro e o último nó devem ser nulos, e a quantidade de elementos deve ser zero.
- Imprima as listas L1, L2 e L3 antes e depois da chamada da função *concatenarListas*, tanto na ordem normal quanto na ordem inversa.

### Exemplo de entrada e saída:

Um exemplo de entrada e saída são apresentados a seguir. O programa desenvolvido deve ser capaz de produzir a saída no formato mostrado.

```
#include <iostream>
using namespace std;
#include "lista_dupla.hpp"
int main(void)
{
 Lista l1 = Lista();
 Lista l2 = Lista();
 Lista l3 = Lista();

 cout << "\n\n###TESTE 1\n";
 insereInicioL(&l1, 5);
 insereInicioL(&l1, 7);
 insereInicioL(&l1, 3);
 insereInicioL(&l1, 1);
 insereInicioL(&l1, 9);
 insereInicioL(&l2, 5);
 insereInicioL(&l2, 4);
 insereInicioL(&l2, 11);
 mostraL(&l1, "l1 normal :");
 mostrarInversoL(&l1, "l1 invertida:");
 mostraL(&l2, "l2 normal :");
 mostrarInversoL(&l2, "l2 invertida:");
 mostraL(&l3, "l3 normal :");
 mostrarInversoL(&l3, "l3 invertida:");
 cout << "Tamanho da lista 3: " << l3.tamanho << endl;

 cout << "\n\n###TESTE 2\n";
 concatenarListas(&l1, &l2, &l3);
 mostraL(&l1, "l1 normal :");
```

```

mostrarInversoL(&l1, "l1 invertida:");
mostraL(&l2, "l2 normal :");
mostrarInversoL(&l2, "l2 invertida:");
mostraL(&l3, "l3 normal :");
mostrarInversoL(&l3, "l3 invertida:");
cout << "Tamanho da lista 3: " << l3.tamanho << endl;

cout << "\n\n###TESTE 3\n";
destróiL(&l1);
destróiL(&l2);
destróiL(&l3);
insereInicioL(&l1, 5);
insereInicioL(&l1, 7);
insereInicioL(&l2, 5);
insereInicioL(&l2, 4);
insereInicioL(&l2, 11);
insereInicioL(&l2, 2);
insereInicioL(&l2, 13);
insereInicioL(&l2, 15);
insereInicioL(&l2, 6);

mostraL(&l1, "l1 normal :");
mostrarInversoL(&l1, "l1 invertida:");
mostraL(&l2, "l2 normal :");
mostrarInversoL(&l2, "l2 invertida:");
mostraL(&l3, "l3 normal :");
mostrarInversoL(&l3, "l3 invertida:");
cout << "Tamanho da lista 3: " << l3.tamanho << endl;

cout << "\n\n###TESTE 4\n";
concatenarListas(&l1, &l2, &l3);
mostraL(&l1, "l1 normal :");
mostrarInversoL(&l1, "l1 invertida:");
mostraL(&l2, "l2 normal :");
mostrarInversoL(&l2, "l2 invertida:");
mostraL(&l3, "l3 normal :");
mostrarInversoL(&l3, "l3 invertida:");
cout << "Tamanho da lista 3: " << l3.tamanho << endl;

destróiL(&l1);
destróiL(&l2);

```

```
 destroiL(&l3);

 return EXIT_SUCCESS;
}
```

```
###TESTE 1
l1 normal :{9, 1, 3, 7, 5}
l1 invertida:{5, 7, 3, 1, 9}
l2 normal :{11, 4, 5}
l2 invertida:{5, 4, 11}
l3 normal :{}
l3 invertida:{}
Tamanho da lista 3: 0

###TESTE 2
l1 normal :{}
l1 invertida:{}
l2 normal :{}
l2 invertida:{}
l3 normal :{9, 1, 3, 7, 5, 11, 4, 5}
l3 invertida:{5, 4, 11, 5, 7, 3, 1, 9}
Tamanho da lista 3: 8

###TESTE 3
l1 normal :{7, 5}
l1 invertida:{5, 7}
l2 normal :{6, 15, 13, 2, 11, 4, 5}
l2 invertida:{5, 4, 11, 2, 13, 15, 6}
l3 normal :{}
l3 invertida:{}
Tamanho da lista 3: 0

###TESTE 4
l1 normal :{}
l1 invertida:{}
l2 normal :{}
l2 invertida:{}
l3 normal :{7, 5, 6, 15, 13, 2, 11, 4, 5}
l3 invertida:{5, 4, 11, 2, 13, 15, 6, 5, 7}
Tamanho da lista 3: 9
```